Chapter 18 | Worked solutions with Python

Interpreting and Aligning Model Behavior

Work through one score by hand before reading the program. The notebook reads the saved experiment, so it can verify the controls without downloading Qwen.

In [1]

```
from pathlib import Path
import json
import math
import sys

here = Path.cwd().resolve()
root = next(p for p in (here, *here.parents) if
            (p / "examples/ch18_behavior/PRETRAINED_RUN.json").is_file())
if str(root) not in sys.path:
    sys.path.insert(0, str(root))

from examples.ch18_behavior.model import patch_effect, route_policy, summarize

record_path = root / "examples/ch18_behavior/PRETRAINED_RUN.json"
record = json.loads(record_path.read_text())
print(record["model"], "with", len(record["pairs"]), "patching pairs")
```

Output

```
Qwen/Qwen3.5-0.8B-Base with 3 patching pairs
```

1. Recover the odds.

The native update-minus-read gap is $2-4=-2$ and the patched gap is $4-3=1$. The shared softmax denominator cancels in the odds, so the odds of B relative to A change from $e^{-2}=0.1353$ to $e^{1}=2.7183$. If $z_C=6$, then C has the largest logit of the three. Reversing the order of A and B does not establish which of the three is preferred.

In [2]

```
for a, b in ((4, 2), (3, 4)):
    print("gap:", b-a, "odds:", round(math.exp(b-a), 4))
print("top label:", route_policy((3, 4, 6)))
```

Output

```
gap: -2 odds: 0.1353
gap: 1 odds: 2.7183
top label: C
```

2. Measure recovery.

The numerator is $3 - (-2) = 5$. The donor and recipient differ by $2 - (-2) = 4$, so the recovery is $5/4 = 1.25$. The score overshoots the donor. In the second example the numerator still equals 5, but the denominator equals zero. Report the shift of 5 natural-log units and mark the recovery fraction undefined. Native equality does not rule out an effect of the intervention.

In [3]

```
print(patch_effect(-2, 2, 3))
print(patch_effect(-2, -2, 3))
```

Output

```
(5, 1.25)
(5, None)
```

3. Read a feature and change it.

The probe's score changes from 2 to -1. The original model's gap stays at -2 because h_2 stays at 1. For the second intervention hold h_1 at 2 and change h_2 to 0. Then g moves from -2 to 0 while the probe stays at 2. Both comparisons specify the altered coordinate and the downstream outcome.

In [4]

```
for h1, h2 in ((2, 1), (-1, 1), (2, 0)):
    print("vector:", (h1, h2), "probe:", h1, "model gap:", -2*h2)
```

Output

```
vector: (2, 1) probe: 2 model gap: -2
vector: (-1, 1) probe: -1 model gap: -2
vector: (2, 0) probe: 2 model gap: 0
```

4. Check a patch.

A position before the edit has the same causal history in both inputs. Exchanging its vector therefore leaves the score unchanged to numerical precision. After the final block there is no further mixing across token positions. The final normalization and vocabulary readout only use the last vector. Patching any earlier vector at that boundary leaves the gap unchanged. The recorded checks compare both directions.

In [5]

```
blocks = record["pairs"][0]["patches"]
directions = ("read_recipient", "write_recipient")
before = max(abs(layer[key][0]["shift"]) for layer in blocks for key in directions)
earlier_last = max(abs(item["shift"]) for key in directions
                   for item in blocks[-1][key][:-1])
print("largest pre-edit shift:", before)
print("largest earlier-position shift at last block:", earlier_last)
assert max(before, earlier_last) < 0.0006
```

Output

```
largest pre-edit shift: 0.0
largest earlier-position shift at last block: 0.0
```

5. Compare two rewards.

Always reading has four valid labels and two correct proposals per four requests. The other policy has three valid labels and three correct proposals per four requests. A format-only reward prefers the first policy, while the action score prefers the second.

In [6]

```
for proposals in ("AAAA", ("A", "B", "C", "invalid")):
    result = summarize("ABCA", proposals)
    print("format:", result["valid"]/4, "action:", result["correct"]/4)
```

Output

```
format: 1.0 action: 0.5
format: 0.75 action: 0.75
```

6. Apply a threshold.

Subtract 2 and exponentiate to get $(1, e^{-1}, e^{-2})$. Dividing by their sum gives $q(A) = 0.6652$, $q(B) = 0.2447$ and $q(C) = 0.0900$. The constraint alone chooses A. The threshold rule chooses C since $0.6652 < 0.80$. In this case asking delays a request that was already clear.

In [7]

```
logits = (2, 1, 0)
weights = [math.exp(z - max(logits)) for z in logits]
print("restricted shares:", [round(w/sum(weights), 4) for w in weights])
print("decisions:", route_policy(logits), route_policy(logits, 0.8))
```

Output

```
restricted shares: [0.6652, 0.2447, 0.09]
decisions: A C
```

7. Test a shortcut.

For the first pair, compare *I would like to change the saved address* (B) with *Show the saved address before I decide whether to change it* (A). Both contain *change*. For the second pair, compare *Use the new address on my invoice* (B) with *Which address is on file?* (A). Neither contains the cue. Add ambiguous cases with and without the word, for example *I need help with the change form*.

Group templates and their paraphrases by wording family and keep a whole family in one split. Report correct clarification among ambiguous cases and unnecessary clarification among clear cases. Count mistaken update proposals separately so they are visible even when rare.

In [8]

```
targets = ("B", "A", "B", "A", "C")
keyword_policy = ("B", "B", "A", "A", "B")
print(summarize(targets, keyword_policy))
```

Output

```
{'n': 5, 'valid': 5, 'correct': 2, 'right_to_clarify': 0, 'ambiguous': 1, 'incorrect_write': 2}
```

8. Change the decision cost.

The read and update costs stay at 1.05 and 6.00. The new cost of asking is $2(0.60) + 2(0.25) + 0(0.15) = 1.70$, so reading now minimizes expected cost. A real desk would need evidence on the service cost of asking, causing delays and making unintended updates. It would also need calibrated intent estimates for the relevant users and permission rules for any write.

In [9]

```
probabilities = (0.60, 0.25, 0.15)
costs = {"A": (0, 3, 2), "B": (8, 0, 8), "C": (2, 2, 0)}
expected = {action: round(sum(p*c for p, c in zip(probabilities, row, strict=True)), 2)
            for action, row in costs.items()}
print(expected)
print("lowest cost:", min(expected, key=expected.get))
```

Output

```
{'A': 1.05, 'B': 6.0, 'C': 1.7}
lowest cost: A
```

9. Read a head's write.

The first token receives most of the weight. The weighted contributions along the chosen direction are $0.9(-1) = -0.9$ and $0.1(8) = 0.8$. Their sum is -0.1. The second token supplies the positive contribution. This calculation applies to one chosen direction in the projected write.

In [10]

```
weights = (0.9, 0.1)
projected = (-1, 8)
contributions = [a*v for a, v in zip(weights, projected, strict=True)]
print("Writes:", contributions, "sum:", round(sum(contributions), 4))
```

Output

```
Writes: [-0.9, 0.8] sum: -0.1
```

10. Integrate a saturated gradient.

The derivative is $3 - 4u$. The endpoint contribution at $u = 1$ is $(-1)(1 - 0) = -1$. The integrated attribution is $F(1) - F(0) = 1$. The midpoint derivatives are 2 and 0, so they average to 1. The endpoint and path scores have opposite signs.

In [11]

```
def derivative(u):
    return 3 - 4*u

print("Endpoint:", derivative(1))
print("Midpoint average:", (derivative(0.25) + derivative(0.75)) / 2)
print("Endpoint difference:", (3*1 - 2*1**2) - 0)
```

Output

```
Endpoint: -1
Midpoint average: 1.0
Endpoint difference: 1
```

11. Fit a probe by hand.

The centered cross-product is $X^\top y = 1$ and $X^\top X = 2$. The coefficient is $1/(2 + 2) = 0.25$. At 0.8 the centered prediction is 0.20. Add the label mean 0.5, for a score of 0.70 and an update decision at threshold 0.5. A possible test family is requests that quote the word *update* while explicitly asking to read. All their templates should stay outside training.

In [12]

```
width = 1 / (2 + 2)
score = 0.5 + width*0.8
print("Coefficient:", width)
print("Test score:", score)
```

Output

```
Coefficient: 0.25
Test score: 0.7
```

12. Check two routes.

Removing either route changes the gap by $4 - 3 = 1$. Removing both changes it by $4 - (-1) = 5$. The interaction is $4 - 3 - 3 - 1 = -3$. The largest departure from the full model comes when both routes are removed. The effects of the single removals sum to only 2.

In [13]

```
g11, g10, g01, g00 = 4, 3, 3, -1
print("Individual effects:", g11-g01, g11-g10)
print("Joint effect:", g11-g00)
print("Interaction:", g11-g10-g01+g00)
```

Output

```
Individual effects: 1 1
Joint effect: 5
Interaction: -3
```

13. Pay for a larger code.

The one-feature code costs $0.18 + \lambda$, while the two-feature code costs 1.6λ. Equating them gives $0.18 = 0.6\lambda$, so $\lambda = 0.30$. At 0.4 the losses are 0.58 and 0.64. The one-feature code pays more reconstruction error and wins because activating the second feature costs more than repairing that error.

In [14]

```
crossing = 0.18 / 0.6
penalty = 0.4
print("Equal-loss penalty:", round(crossing, 2))
print("One feature:", 0.18 + penalty)
print("Two features:", round(1.6*penalty, 2))
```

Output

```
Equal-loss penalty: 0.3
One feature: 0.5800000000000001
Two features: 0.64
```

14. Rescore a policy.

With $\beta = 0.5$, the raw weights are 0.5, 0.3, and $0.2e^2$. Divide by about 2.278 to get 0.2195, 0.1317, and 0.6488. Adding 2 to every reward multiplies all three raw weights by e^4. The same factor appears in the denominator, leaving the policy fixed.

In [15]

```
from examples.ch18_behavior.feedback import preference_probability, reward_tilt

reference = [math.log(x) for x in (0.5, 0.3, 0.2)]
for reward in ((0, 0, 1), (2, 2, 3)):
    result = reward_tilt(reference, reward, 0.5)
    print("Policy:", [round(value, 4) for value in result])
```

Output

```
Policy: [0.2195, 0.1317, 0.6488]
Policy: [0.2195, 0.1317, 0.6488]
```

15. Separate preference and policy scores.

A reward difference of 1 predicts preference probability 0.7311, and the loss for the first answer is about 0.3133. Both stay fixed under a common shift. DPO first compares policy odds to reference odds. The margin is $0.5(\ln 4 - \ln 2) = 0.3466$ and the pair probability is about 0.5858. The first predicts a rater's pairwise choice using the supplied rewards. The second represents the same kind of pair judgment through log odds relative to a reference. Neither is the absolute probability that the policy emits the first answer.

In [16]

```
p = preference_probability(1.5, 0.5)
print("Preference probability:", round(p, 4), "loss:", round(-math.log(p), 4))
print("Shifted score:", round(preference_probability(3.5, 2.5), 4))
margin = 0.5*(math.log(4)-math.log(2))
print("DPO margin:", round(margin, 4),
      "preference probability:", round(preference_probability(margin, 0), 4))
```

Output

```
Preference probability: 0.7311 loss: 0.3133
Shifted score: 0.7311
DPO margin: 0.3466 preference probability: 0.5858
```

16. Plan an audit.

Solve $(1 - p)^{200} = 0.05$ to get $p = 1 - 0.05^{1/200} \approx 0.0149$. Under the assumptions, the upper bound is about 1.49 percent. A challenge set seeks cases that break the system, for example a clear update request with an unauthorized account. An independently sampled audit can estimate how frequent failures are in its stated population. The challenge set gives no sampling rate for that population.

In [17]

```
upper = 1 - 0.05 ** (1/200)
print("95 percent upper bound:", round(100*upper, 3), "percent")
```

Output

```
95 percent upper bound: 1.487 percent
```